

Roteiro de Laboratório — Formatação, Apresentação e Tratamento de Dados no SELECT

Curso de Banco de Dados Relacional aplicado à Fiscalização Tributária Módulo 11 — A lista de seleção como camada de apresentação SGBD: Microsoft SQL Server (T-SQL) — Banco: curso_integridade_fiscal

Como usar este roteiro

Este é um **roteiro de laboratório**, não um texto para leitura passiva. Cada etapa segue sempre a mesma estrutura:

O que fazer → a consulta a executar **O que observar** → o ponto que a consulta demonstra **A armadilha** → o erro que essa função costuma provocar em trabalho fiscal

Execute cada consulta no SSMS, olhe o resultado e só então passe adiante. O tempo estimado é de **3 a 4 horas** com os exercícios.

Preparação

```
USE curso_integridade_fiscal;
GO
```

O laboratório usa o banco montado pelos scripts da pasta **banco_fiscal/**. Uma tabela adicional é necessária — o quadro societário, que traz o **CPF** (o restante do modelo só tem CNPJ):

```
-- Execute uma única vez, se ainda não executou:
-- banco_fiscal\09_complemento_socios.sql
SELECT COUNT(*) AS socios FROM dbo.socio_administrador; -- deve retornar 138
```

Aviso sobre os resultados exibidos. Os valores mostrados nos quadros de "resultado esperado" vêm da base gerada pelos scripts do curso, com semente fixa. Se você regenerou os dados com outros parâmetros, os números mudam — a estrutura da resposta, não.

Etapa 0 — Por que formatar no SQL?

Antes da primeira função, uma decisão de projeto que o auditor precisa tomar conscientemente:

Formatar no SQL	Formatar na ferramenta (Excel, Power BI, relatório)
A saída já sai pronta para colar em ofício, e-mail ou processo	O dado permanece numérico/data, permitindo ordenar, somar e filtrar corretamente
Todos que executarem a consulta veem exatamente o mesmo formato	Cada usuário aplica o formato que precisa
Transforma número em texto — e texto ordena como texto: '9.000,00' vem depois de '10.000,00'	Preserva o tipo e, portanto, a ordenação e a agregação
Custa CPU no servidor, que é recurso compartilhado	Custa na máquina do usuário

Regra prática adotada neste curso:

- 1. **Consulta intermediária** (que alimenta outra consulta, uma view, o Power Query ou o Power BI): **não formate**. Devolva **DECIMAL**, **DATE**, **CHAR**. Formatação ali quebra junção, ordenação e *query folding*.
- 2. **Consulta final de apresentação** (a listagem que vai para o ofício, para o auto de infração ou para a tela): **formate à vontade**, porque ali o destino é o olho humano.

3. **Nunca formate no WHERE.** Aplicar função à coluna filtrada impede o uso de índice (predicado não *sargable*) — assunto do Módulo 10, seção 10.

Guarde esta distinção: ela reaparece em todas as etapas.

Etapa 1 — Arredondamento e truncamento

1.1 ROUND, CEILING, FLOOR e o terceiro argumento de ROUND

O que fazer:

```
SELECT n.chave_acesso,
       n.valor_total,
       ROUND(n.valor_total, 0)           AS arredondado_inteiro,
       ROUND(n.valor_total, -3)          AS arredondado_milhar,
       CEILING(n.valor_total)            AS teto,
       FLOOR(n.valor_total)              AS piso,
       ROUND(n.valor_total, 0, 1)        AS truncado           -- 3o argumento <> 0 = TRUNCA
FROM   dbo.nfe AS n
WHERE  n.id_nfe <= 5
ORDER BY n.id_nfe;
```

O que observar: `ROUND(valor, 0)` devolve `13640.00` para `13640.16` — mantém as casas decimais do **tipo**, apenas zeradas. Quem quer o inteiro precisa converter: `CAST(ROUND(valor,0) AS INT)`.

O **terceiro argumento** é o detalhe que quase ninguém conhece: quando diferente de zero, **ROUND trunca** em vez de arredondar. `ROUND(13640.16, 0, 1) = 13640.00`; `ROUND(13640.96, 0, 1) = 13640.00` também.

A armadilha — arredondar não é truncar, e a diferença vira valor de auto:

```
-- Uma casa decimal a mais ou a menos, multiplicada por 30 mil itens
SELECT SUM(ROUND(i.valor_total_item * 0.18, 2)) AS icms_arredondado,
       SUM(ROUND(i.valor_total_item * 0.18, 2, 1)) AS icms_truncado,
       SUM(ROUND(i.valor_total_item * 0.18, 2))
       - SUM(ROUND(i.valor_total_item * 0.18, 2, 1)) AS diferenca
FROM   dbo.nfe_item AS i;
```

A diferença é de centavos por item — e de milhares de reais no somatório. Em um auto de infração, o critério de arredondamento precisa estar declarado e ser o mesmo da legislação aplicável. **Nunca deixe implícito.**

1.2 Onde arredondar: antes ou depois de somar?

O que fazer:

```
SELECT ROUND(SUM(i.valor_total_item * 0.18), 2) AS soma_depois_arredonda,
       SUM(ROUND(i.valor_total_item * 0.18, 2)) AS arredonda_depois_soma
FROM   dbo.nfe_item AS i;
```

O que observar: os dois números são diferentes. O primeiro é matematicamente mais preciso; o segundo reproduz o que acontece **documento a documento** no mundo real, onde cada nota já foi arredondada na emissão. Para conferir escrituração, o segundo costuma ser o correto — porque é o que o contribuinte fez.

1.3 Divisão inteira e o tipo do resultado

O que fazer:

```

SELECT 7 / 2 AS divisao_inteira, -- 3 (!)
       7.0 / 2 AS com_decimal, -- 3.500000
       CAST(7 AS DECIMAL(10,2)) / 2 AS com_cast, -- 3.50
       COUNT(*) AS qtd_notas,
       SUM(CASE WHEN situacao='CANCELADA' THEN 1 ELSE 0 END) AS canceladas,
       SUM(CASE WHEN situacao='CANCELADA' THEN 1 ELSE 0 END) / COUNT(*) AS pct_errado,
       CAST(SUM(CASE WHEN situacao='CANCELADA' THEN 1 ELSE 0 END) AS DECIMAL(12,4))
         / NULLIF(COUNT(*), 0) * 100 AS pct_correto
FROM dbo.nfe;

```

A armadilha: `pct_errado` retorna **zero**. Dois inteiros divididos produzem inteiro, e a parte fracionária é descartada antes de qualquer multiplicação. Todo percentual calculado sobre contagens precisa de `CAST(... AS DECIMAL(...))` **antes** da divisão. Este é, disparado, o erro de formatação mais comum em relatórios de fiscalização.

1.4 Por que `DECIMAL` e nunca `FLOAT` em valor monetário

```

SELECT CAST(0.1 AS FLOAT) + CAST(0.2 AS FLOAT) AS soma_float, -- 0.30000000000000004
       CAST(0.1 AS DECIMAL(10,2)) + CAST(0.2 AS DECIMAL(10,2)) AS soma_decimal; -- 0.30

```

`FLOAT` é binário e não representa exatamente frações decimais. Em lançamento fiscal, isso produz diferenças de centavos que ninguém consegue explicar em impugnação. O modelo do curso usa `DECIMAL(15,2)` justamente por isso.

Etapa 2 — Formatação numérica

2.1 `FORMAT` com cultura brasileira

O que fazer:

```

SELECT n.valor_total,
       FORMAT(n.valor_total, 'N2', 'pt-BR') AS numero_br, -- 13.640,16
       FORMAT(n.valor_total, 'C', 'pt-BR') AS moeda_br, -- R$ 13.640,16
       FORMAT(n.valor_total, '#,##0.00', 'pt-BR') AS mascara_explicita,
       FORMAT(n.valor_icms / NULLIF(n.valor_total,0), 'P2', 'pt-BR') AS carga_percentual
FROM dbo.nfe AS n
WHERE n.id_nfe <= 5;

```

O que observar: o terceiro argumento (a *cultura*) é o que inverte ponto e vírgula. Sem ele, o SQL Server usa a cultura da sessão — que no servidor costuma ser `en-US`, produzindo `13,640.16`. Em material que vai para processo administrativo, o formato precisa ser o brasileiro **explicitamente**, não por sorte de configuração.

Formatos numéricos padrão mais úteis:

Formato	Resultado (pt-BR)	Uso
'N2'	13.640,16	valores em geral
'N0'	13.640	quantidades
'C' / 'C2'	R\$ 13.640,16	moeda
'P2'	21,66%	percentual (multiplica por 100 automaticamente)
'0000'	0042	preenchimento com zeros à esquerda
'#,##0.00'	13.640,16	máscara explícita

Atenção ao P: `FORMAT(0.2166, 'P2')` devolve `21,66%`. Se você já multiplicou por 100, vai exibir `2.166,00%`. Escolha um dos dois caminhos.

2.2 A alternativa rápida: `CONVERT` com `MONEY`

O que fazer:

```
SELECT TOP (5)
    n.valor_total,
    FORMAT(n.valor_total, 'N2', 'pt-BR') AS via_format,
    REPLACE(REPLACE(REPLACE(CONVERT(VARCHAR(30), CAST(n.valor_total AS MONEY), 1),
        ',', '|'), '.', ','), '|', '.') AS via_convert
FROM    dbo.nfe AS n
ORDER BY n.id_nfe;
```

O que observar: os dois produzem `13.640,16`. O caminho com `CONVERT` parece feio — e é —, mas roda **muito mais rápido**: `FORMAT` é implementado em CLR e chega a ser uma ordem de grandeza mais lento. A troca tripla de `REPLACE` funciona assim: `CONVERT(..., 1)` devolve `13,640.16` no padrão americano; trocamos a vírgula por um caractere neutro, o ponto por vírgula e, por fim, o neutro por ponto.

Meça você mesmo:

```
SET STATISTICS TIME ON;
SELECT COUNT(*) FROM (SELECT FORMAT(valor_total, 'N2', 'pt-BR') AS v FROM dbo.nfe) AS t;
SELECT COUNT(*) FROM (SELECT CONVERT(VARCHAR(30), CAST(valor_total AS MONEY), 1) AS v FROM dbo.nfe)
AS t;
SET STATISTICS TIME OFF;
```

A regra: `FORMAT` em listagem final de algumas centenas de linhas, sem problema. `FORMAT` sobre milhões de linhas de `nfe_item`, jamais.

2.3 Alinhamento de colunas em saída de texto

Quando o resultado vai para um arquivo `.txt` de largura fixa (ainda comum em intimações e em intercâmbio com sistemas legados):

```
SELECT RIGHT(REPLICATE(' ', 16) + FORMAT(n.valor_total, 'N2', 'pt-BR'), 16) AS valor_alinhado,
    LEFT(c.razao_social + REPLICATE(' ', 40), 40) AS razao_40,
    RIGHT(REPLICATE('0', 9) + CAST(n.numero AS VARCHAR(9)), 9) AS numero_zeros
FROM    dbo.nfe AS n
    INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE   n.id_nfe <= 8;
```

O que observar: o padrão `RIGHT(REPLICATE(caractere, n) + texto, n)` é a forma clássica de preencher à esquerda; `LEFT(texto + REPLICATE(' ', n), n)` preenche à direita. No SQL Server 2022 há `LPAD/ RPAD` nativos, mas o padrão acima funciona em qualquer versão. Para ver o alinhamento, mude a saída para texto com `Ctrl+T`.

Etapa 3 — Formatação de datas

3.1 `CONVERT` com estilo — o caminho tradicional

O que fazer:

```
SELECT n.data_emissao,
    CONVERT(VARCHAR(10), n.data_emissao, 103) AS dd_mm_aaaa, -- 07/04/2025
    CONVERT(VARCHAR(10), n.data_emissao, 102) AS aaaa_mm_dd, -- 2025.04.07
```

```

    CONVERT(VARCHAR(8), n.data_emissao, 112) AS aaaammdd,      -- 20250407
    CONVERT(VARCHAR(10), n.data_emissao, 23) AS iso,          -- 2025-04-07
    CONVERT(VARCHAR(6), n.data_emissao, 112) AS periodo_efd    -- 202504
FROM   dbo.nfe AS n
WHERE  n.id_nfe <= 5;

```

O que observar: o estilo **112** (AAAAMDD) é o mais útil na fiscalização, por dois motivos: é o formato do SPED e é o único literal de data **imune** a qualquer configuração de idioma. `CONVERT(VARCHAR(6), data, 112)` entrega direto o período de apuração AAAAMM usado em `efd_c100`.

Estilos que valem memorizar:

Estilo	Resultado	Observação
103	07/04/2025	padrão britânico/brasileiro
112	20250407	ISO compacto — SPED e literais seguros
23	2025-04-07	ISO com hífen
120	2025-04-07 00:00:00	ODBC canônico
108	14:35:22	só a hora

3.2 FORMAT para datas e nomes em português

```

SELECT n.data_emissao,
       FORMAT(n.data_emissao, 'dd/MM/yyyy') AS br,
       FORMAT(n.data_emissao, 'yyyyMM') AS periodo,
       FORMAT(n.data_emissao, 'MMMM', 'pt-BR') AS mes_extenso,
       FORMAT(n.data_emissao, 'MMM/yy', 'pt-BR') AS mes_curto,
       FORMAT(n.data_emissao, 'dddd', 'pt-BR') AS dia_semana,
       FORMAT(n.data_emissao, 'dd ''de'' MMMM ''de'' yyyy', 'pt-BR') AS por_extenso
FROM   dbo.nfe AS n
WHERE  n.id_nfe <= 5;

```

O que observar: MM é mês e mm é minuto; dd é dia e dddd é o nome do dia da semana. Trocar a caixa das letras muda o significado — erro clássico. A cultura 'pt-BR' é o que produz *abril* em vez de *April*.

A data por extenso serve para o preâmbulo de documentos: "Lavrado em 07 de abril de 2025".

3.3 Aritmética de datas para o trabalho fiscal

```

SELECT os.numero_os,
       os.data_abertura,
       os.data_conclusao,
       DATEDIFF(DAY, os.data_abertura, COALESCE(os.data_conclusao, GETDATE())) AS dias_corridos,
       DATEADD(DAY, 30, os.data_abertura) AS prazo_30_dias,
       EOMONTH(os.data_abertura) AS ultimo_dia_do_mes,
       EOMONTH(os.data_abertura, -1) AS ultimo_dia_mes_anterior,
       DATEPART(QUARTER, os.data_abertura) AS trimestre,
       DATENAME(MONTH, os.data_abertura) AS nome_mes_idioma_da_sessao
FROM   dbo.ordem_servico AS os
WHERE  os.id_os <= 6;

```

O que observar: `DATENAME` depende do idioma da **sessão** (`SET LANGUAGE Portuguese`; muda o resultado); `FORMAT(..., 'MMMM', 'pt-BR')` não depende — é explícito. Prefira o explícito.

`EOMONTH` resolve de uma vez o cálculo de fim de período de apuração, incluindo fevereiro bissexto.

3.4 Faixas de prazo com data — antecipando o CASE

```
SELECT os.numero_os,
       FORMAT(os.data_abertura, 'dd/MM/yyyy') AS abertura,
       DATEDIFF(DAY, os.data_abertura, GETDATE()) AS dias,
       CASE WHEN os.data_conclusao IS NOT NULL THEN 'CONCLUIDA'
            WHEN DATEDIFF(DAY, os.data_abertura, GETDATE()) > 180 THEN 'CRITICA (+180 dias)'
            WHEN DATEDIFF(DAY, os.data_abertura, GETDATE()) > 90 THEN 'ATENCAO (+90 dias)'
            ELSE 'NO PRAZO'
       END AS faixa_de_prazo
FROM   dbo.ordem_servico AS os
ORDER BY dias DESC;
```

3.5 A armadilha do intervalo de datas

```
-- ERRADO em coluna DATETIME: perde os documentos emitidos ao longo do dia 31
WHERE data_emissao BETWEEN '2025-01-01' AND '2025-01-31'

-- CORRETO e sargable
WHERE data_emissao >= '2025-01-01' AND data_emissao < '2025-02-01'

-- ERRADO por outro motivo: mata o índice
WHERE YEAR(data_emissao) = 2025 AND MONTH(data_emissao) = 1

-- ERRADO e imprevisível: depende de SET DATEFORMAT
WHERE data_emissao = '01/02/2025'
```

Regra: literal de data sempre em 'AAAA-MM-DD' ou 'AAAAMMDD'; intervalo sempre com >= e <; nunca aplique função sobre a coluna filtrada.

Etapa 4 — Conversões seguras

4.1 CAST, CONVERT e a diferença entre eles

```
SELECT CAST('20250407' AS DATE)           AS cast_data,
       CONVERT(DATE, '20250407')          AS convert_data,
       CONVERT(VARCHAR(10), GETDATE(), 103) AS convert_com_estilo,
       CAST(13640.16 AS INT)              AS cast_int;      -- 13640 (trunca!)
```

CAST é ANSI e portátil; **CONVERT** é específico do T-SQL e aceita o **estilo** — que é justamente o que se precisa para datas. Use **CAST** quando o estilo não importa, **CONVERT** quando importa.

CAST para inteiro **trunca**, não arredonda. **CAST(13640.96 AS INT) = 13640**.

4.2 TRY_CAST e TRY_CONVERT — converter sem derrubar o lote

O que fazer:

```
-- Sem TRY, um único valor invalido aborta a consulta inteira
SELECT CAST(cpf AS BIGINT) FROM dbo.socio_administrador; -- ERRO: conversion failed

-- Com TRY, o valor invalido vira NULL e o restante continua
SELECT s.id_socio,
       s.cpf AS cpf_original,
       TRY_CAST(s.cpf AS BIGINT) AS cpf_numerico,
```

```

CASE WHEN TRY_CAST(s.cpf AS BIGINT) IS NULL THEN 'NAO CONVERSIVEL'
      ELSE 'OK' END AS resultado
FROM   dbo.socio_administrador AS s
ORDER BY resultado, s.id_socio;

```

O que observar: os CPF gravados com máscara (942.513.327-17) não convertem — e é exatamente isso que se quer descobrir. `TRY_CAST` transforma "erro que aborta" em "coluna que se pode contar":

```

SELECT COUNT(*) AS total,
       COUNT(TRY_CAST(cpf AS BIGINT)) AS conversiveis,
       COUNT(*) - COUNT(TRY_CAST(cpf AS BIGINT)) AS com_mascara_ou_sujeira
FROM   dbo.socio_administrador;

```

Esse é o **diagnóstico de qualidade de dado** que deve preceder qualquer cruzamento por CPF.

4.3 Convertendo o período da EFD em data

```

SELECT DISTINCT
    e.periodo_apuracao,
    CONVERT(DATE, e.periodo_apuracao + '01') AS primeiro_dia,
    EOMONTH(CONVERT(DATE, e.periodo_apuracao + '01')) AS ultimo_dia,
    FORMAT(CONVERT(DATE, e.periodo_apuracao + '01'), 'MMM/yyyy', 'pt-BR') AS competencia
FROM   dbo.efd_c100 AS e
ORDER BY e.periodo_apuracao;

```

O que observar: `'202501' + '01'` produz `'20250101'`, que é o literal ISO compacto — converte sem ambiguidade. Se algum período estiver corrompido, troque por `TRY_CONVERT` e conte os nulos antes de prosseguir.

4.4 Conversão implícita: o risco silencioso

```

-- Funciona, mas o SQL Server converte a coluna inteira para comparar
SELECT COUNT(*) FROM dbo.contribuinte WHERE cnpj = 27712979000187;      -- numero!

-- Correto: compara texto com texto
SELECT COUNT(*) FROM dbo.contribuinte WHERE cnpj = '27712979000187';

```

A armadilha: na primeira consulta, a precedência de tipos manda converter `cnpj` (texto) para número — o que impede o uso do índice e, pior, provoca erro quando algum registro não for numérico. Compare sempre tipos iguais. O CNPJ é **texto**, não número: tem zeros à esquerda e não se soma.

Etapa 5 — Strings, máscaras e a limpeza do CPF e do CNPJ

5.1 As funções de string essenciais

```

SELECT s.id_socio,
       '[' + s.nome + ']' AS nome_bruto,
       '[' + TRIM(s.nome) + ']' AS sem_espacos,
       '[' + UPPER(TRIM(s.nome)) + ']' AS normalizado,
       LEN(s.nome) AS len_ignora_espaco_a_direita,
       DATALENGTH(s.nome) AS bytes_reais,
       LEFT(TRIM(s.nome), CHARINDEX(' ', TRIM(s.nome) + ' ') - 1) AS primeiro_nome
FROM   dbo.socio_administrador AS s

```

```
WHERE s.nome <> UPPER(TRIM(s.nome))
ORDER BY s.id_socio;
```

O que observar: `LEN` ignora espaços à direita; `DATALENGTH` não. É por isso que uma coluna `CHAR(44)` com 43 caracteres devolve `LEN = 43` — o que permitiu o exercício de chave de acesso do Módulo 9.

`TRIM` (SQL Server 2017+) substitui `LTRIM(RTRIM(...))`. Em versões anteriores, use a forma composta.

5.2 Aplicar máscara de CNPJ

Duas formas, ambas válidas:

```
SELECT TOP (10)
    c.cnpj AS cnpj_limpo,
    -- Forma 1: concatenação com SUBSTRING (funciona em qualquer versão)
    SUBSTRING(c.cnpj,1,2) + '.' + SUBSTRING(c.cnpj,3,3) + '.' +
    SUBSTRING(c.cnpj,6,3) + '/' + SUBSTRING(c.cnpj,9,4) + '-' +
    SUBSTRING(c.cnpj,13,2) AS cnpj_mascara_1,
    -- Forma 2: FORMAT sobre valor numérico (mais curta, porém mais lenta)
    FORMAT(CAST(c.cnpj AS BIGINT), '00\000\000\0000\00') AS cnpj_mascara_2,
    -- Forma 3: STUFF, inserindo os separadores de trás para frente
    STUFF(STUFF(STUFF(STUFF(c.cnpj, 13, 0, '-'), 9, 0, '/'), 6, 0, '.'), 3, 0, '.') AS
cnpj_mascara_3,
    c.razao_social
FROM    dbo.contribuinte AS c
ORDER BY c.id_contribuinte;
```

Resultado esperado (primeira linha):

cnpj_limpo	cnpj_mascara_1	razao_social
27712979000187	27.712.979/0001-87	ALFA DISTRIBUIDORA DE BEBIDAS EIRELI

O que observar em `STUFF`: a função tem a assinatura `STUFF(texto, posição, quantos_apagar, o_que_inserir)`. Com 0 no terceiro argumento, ela **insere sem apagar**. As inserções são feitas **da direita para a esquerda** (13, 9, 6, 3) porque cada inserção desloca as posições seguintes — inserir da esquerda para a direita exigiria recalcular todas as posições.

A armadilha da forma 2: `CAST(cnpj AS BIGINT)` **perde o zero à esquerda**. Para o CNPJ do modelo, que sempre tem 14 dígitos significativos, funciona; para CPF, quebra (ver 5.4).

5.3 Remover máscara — a normalização que precede todo cruzamento

```
SELECT s.id_socio,
    s.cpf AS original,
    REPLACE(REPLACE(REPLACE(TRIM(s.cpf), '.', ''), '-', ''), ' ', '') AS so_digitos,
    LEN(REPLACE(REPLACE(REPLACE(TRIM(s.cpf), '.', ''), '-', ''), ' ', '')) AS qtd_digitos
FROM    dbo.socio_administrador AS s
ORDER BY qtd_digitos, s.id_socio;
```

O que observar: aparecem registros com **10 e até 8 dígitos**. Não são erros de digitação: são CPF que, em algum ponto do caminho, foram gravados como número e perderam os zeros à esquerda. É o defeito de dado mais frequente em base cadastral brasileira.

5.4 A limpeza completa do CPF, em uma expressão

```
WITH limpo AS (
    SELECT s.id_socio,
        s.id_contribuinte,
```



```

        UPPER(TRIM(s.nome)) AS nome,
        RIGHT('0000000000' + REPLACE(REPLACE(REPLACE(TRIM(s.cpf), '.', ''), '-', ''), ' ', ''),
11) AS cpf
    FROM    dbo.socio_administrador AS s
)
SELECT l.id_socio,
       l.nome,
       l.cpf
       AS cpf_normalizado,
       SUBSTRING(l.cpf,1,3) + '.' + SUBSTRING(l.cpf,4,3) + '.' +
       SUBSTRING(l.cpf,7,3) + '-' + SUBSTRING(l.cpf,10,2)
       AS cpf_formatado,
       c.razao_social
FROM    limpo AS l
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = l.id_contribuinte
ORDER BY l.id_socio;

```

O que observar: o padrão `RIGHT('0000000000' + digitos, 11)` recompõe os zeros perdidos — concatena onze zeros à esquerda e pega os onze últimos caracteres. É a mesma técnica do preenchimento visto em 2.3, aqui com finalidade de **integridade**, não de estética.

Por que isso importa na fiscalização: sem a normalização, o mesmo sócio aparece como três pessoas diferentes (942.513.327-17, 94251332717 e 9425133271 7), e o cruzamento que procura o mesmo CPF em vários contribuintes — indício de grupo econômico ou de interposição de pessoas — simplesmente não encontra nada.

5.5 O cruzamento que só funciona depois da limpeza

```

WITH socio_limpo AS (
    SELECT RIGHT('0000000000' + REPLACE(REPLACE(REPLACE(TRIM(cpf), '.', ''), '-', ''), ' ', ''),
11) AS cpf,
           UPPER(TRIM(nome)) AS nome,
           id_contribuinte
    FROM    dbo.socio_administrador
)
SELECT SUBSTRING(sl.cpf,1,3) + '.' + SUBSTRING(sl.cpf,4,3) + '.' +
       SUBSTRING(sl.cpf,7,3) + '-' + SUBSTRING(sl.cpf,10,2) AS cpf,
       MIN(sl.nome) AS nome,
       COUNT(DISTINCT sl.id_contribuinte) AS qtd_empresas,
       STRING_AGG(c.razao_social, ' | ') WITHIN GROUP (ORDER BY c.razao_social) AS empresas
FROM    socio_limpo AS sl
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = sl.id_contribuinte
GROUP BY sl.cpf
HAVING COUNT(DISTINCT sl.id_contribuinte) >= 3
ORDER BY qtd_empresas DESC;

```

Resultado esperado: 6 pessoas físicas que participam de 3 ou 4 empresas cada. Execute a mesma consulta **sem** a normalização (agrupando por `cpf` bruto) e compare: o resultado vem vazio ou incompleto. É a demonstração mais direta de que tratamento de dado não é cosmético.

5.6 Máscaras dos demais campos fiscais

```

SELECT TOP (5)
    n.chave_acesso,
    -- chave em grupos de 4, como aparece no DANFE
    SUBSTRING(n.chave_acesso,1,4) + ' ' + SUBSTRING(n.chave_acesso,5,4) + ' ' +
    SUBSTRING(n.chave_acesso,9,4) + ' ' + SUBSTRING(n.chave_acesso,13,4) + ' ' +
    SUBSTRING(n.chave_acesso,17,4) + ' ' + SUBSTRING(n.chave_acesso,21,4) + ' ' +
    SUBSTRING(n.chave_acesso,25,4) + ' ' + SUBSTRING(n.chave_acesso,29,4) + ' ' +
    SUBSTRING(n.chave_acesso,33,4) + ' ' + SUBSTRING(n.chave_acesso,37,4) + ' ' +
    SUBSTRING(n.chave_acesso,41,4)
    AS chave_danfe,
    i.ncm,

```

```

        STUFF(STUFF(i.ncm, 7, 0, '.'), 5, 0, '.') AS ncm_formatado, --
2204.21.00
        i.cfop,
        STUFF(i.cfop, 2, 0, '.') AS cfop_formatado, -- 5.405
        CONCAT_WS(' - ', i.cfop, i.descricao) AS cfop_descricao
FROM    dbo.nfe AS n
        INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe
ORDER BY n.id_nfe, i.num_item;

```

O que observar: `CONCAT` e `CONCAT_WS` (com separador) **ignoram nulos** — diferentemente do operador `+`, que propaga `NULL` e apaga a linha inteira:

```

SELECT 'A' + NULL + 'B' AS com_operador, -- NULL
       CONCAT('A', NULL, 'B') AS com_concat, -- AB
       CONCAT_WS(' - ', 'A', NULL, 'B') AS com_concat_ws; -- A - B

```

Em endereço, nome fantasia e complemento — campos que aceitam nulo — use sempre `CONCAT`.

Etapa 6 — Um relatório de apresentação completo

Reúna o que foi visto até aqui em uma listagem pronta para anexar a uma ordem de serviço:

```

SELECT TOP (20)
    SUBSTRING(c.cnpj,1,2) + '.' + SUBSTRING(c.cnpj,3,3) + '.' +
    SUBSTRING(c.cnpj,6,3) + '/' + SUBSTRING(c.cnpj,9,4) + '-' +
    SUBSTRING(c.cnpj,13,2) AS [CNPJ],
    UPPER(TRIM(c.razao_social)) AS [Razao Social],
    m.nome AS [Municipio],
    COALESCE(m.regiao_fiscal, 'FORA DA PB') AS [Regiao Fiscal],
    FORMAT(n.data_emissao, 'dd/MM/yyyy') AS [Emissao],
    FORMAT(n.valor_total, 'N2', 'pt-BR') AS [Valor Total],
    FORMAT(n.valor_icms, 'N2', 'pt-BR') AS [ICMS],
    FORMAT(n.valor_icms / NULLIF(n.valor_total, 0), 'P2', 'pt-BR') AS [Carga],
    n.chave_acesso AS [Chave de Acesso]
FROM    dbo.nfe AS n
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
        INNER JOIN dbo.municipio AS m ON m.id_municipio = c.id_municipio
WHERE   n.situacao = 'AUTORIZADA'
        AND n.tipo_operacao = '1'
        AND n.data_emissao >= '2025-01-01' AND n.data_emissao < '2025-02-01'
ORDER BY n.valor_total DESC;

```

O que observar: os aliases entre colchetes com acento e espaço produzem cabeçalhos legíveis ao colar em documento. E note o `ORDER BY n.valor_total DESC` — pela coluna **numérica original**, não pela versão formatada. Ordenar por `[Valor Total]` (texto) colocaria R\$ 9.999,00 depois de R\$ 10.000,00.

Este é o ponto central de toda a etapa: **formate para exibir, ordene e filtre pelo dado bruto**.

Etapa 7 — A expressão `CASE`, explicada

7.1 O que `CASE` é (e o que não é)

`CASE` **não é um comando** — é uma **expressão**. A distinção não é preciosismo: comandos executam ações (`UPDATE`, `DELETE`); expressões **produzem um valor**. Como `CASE` produz um valor escalar, ele pode aparecer em qualquer lugar onde um valor caberia: na lista de seleção, no `WHERE`, no `ORDER BY`, no `GROUP BY`, no `HAVING`, dentro de uma função agregada, no `SET` de um `UPDATE`, no `ON` de uma junção.

É o equivalente, em SQL, ao **se... então... senão** das linguagens de programação — com a diferença fundamental de que **não desvia o fluxo**: ele apenas escolhe qual valor devolver, linha a linha.

7.2 As duas formas

Forma simples — compara uma expressão com valores constantes, por igualdade:

```
SELECT e.ind_oper,
       CASE e.ind_oper
         WHEN '0' THEN 'ENTRADA / AQUISICAO'
         WHEN '1' THEN 'SAIDA / PRESTACAO'
         ELSE 'INDICADOR INVALIDO'
       END AS tipo_operacao,
       COUNT(*) AS qtd
FROM   dbo.efd_c100 AS e
GROUP BY e.ind_oper;
```

Forma pesquisada — avalia condições booleanas quaisquer, na ordem em que foram escritas:

```
SELECT c.razao_social,
       c.faturamento_declarado,
       CASE WHEN c.faturamento_declarado IS NULL THEN 'NAO INFORMADO'
            WHEN c.faturamento_declarado < 360000 THEN 'MICROEMPRESA'
            WHEN c.faturamento_declarado < 4800000 THEN 'PEQUENO PORTE'
            WHEN c.faturamento_declarado < 78000000 THEN 'MEDIO PORTE'
            ELSE 'GRANDE PORTE'
       END AS porte
FROM   dbo.contribuinte AS c
ORDER BY c.faturamento_declarado DESC;
```

A forma simples só compara **igualdade** e não consegue testar **IS NULL** (porque **x = NULL** é UNKNOWN, nunca TRUE). Quando houver nulos ou faixas, use a forma pesquisada.

7.3 As quatro regras que governam o CASE

Regra 1 — a avaliação é sequencial e para na primeira condição verdadeira. Por isso, a ordem das linhas do exemplo acima importa: se **< 78000000** viesse antes de **< 360000**, toda microempresa seria classificada como médio porte. **Escreva sempre da condição mais restritiva para a mais ampla.**

Regra 2 — sem ELSE, o resultado é NULL.

```
SELECT n.situacao,
       CASE n.situacao WHEN 'AUTORIZADA' THEN 'VALIDA' END AS sem_else,
       COUNT(*) AS qtd
FROM   dbo.nfe AS n
GROUP BY n.situacao;
```

As linhas **CANCELADA**, **DENEGADA** e **INUTILIZADA** devolvem **NULL**. Isso pode ser exatamente o que se quer — é a base da contagem condicional da etapa 8 — mas em coluna de relatório costuma ser esquecimento. **Declare o ELSE** sempre que a coluna for para o olho humano.

Regra 3 — todos os ramos precisam devolver o mesmo tipo (ou tipos compatíveis).

```
-- ERRO: "Conversion failed when converting the varchar value 'NAO INFORMADO' to data type int"
SELECT CASE WHEN faturamento_declarado IS NULL THEN 'NAO INFORMADO'
           ELSE faturamento_declarado
```

```

        END
    FROM dbo.contribuinte;

-- CORRETO: converte explicitamente o ramo numérico para texto
SELECT CASE WHEN faturamento_declarado IS NULL THEN 'NAO INFORMADO'
           ELSE FORMAT(faturamento_declarado, 'N2', 'pt-BR')
        END AS faturamento
    FROM dbo.contribuinte;

```

O SQL Server aplica a **precedência de tipos** e tenta converter todos os ramos para o tipo de maior precedência — e **INT** tem precedência sobre **VARCHAR**. O resultado é o erro acima, que confunde quem espera que "texto vence".

Regra 4 — CASE protege contra erro de execução, mas com uma ressalva. O idioma abaixo é o padrão para evitar divisão por zero:

```

SELECT CASE WHEN n.valor_total = 0 THEN 0
           ELSE n.valor_icms / n.valor_total
        END AS carga
    FROM dbo.nfe AS n;

```

Funciona, mas em consultas complexas o otimizador pode reordenar operações e a proteção falhar. A forma **sempre segura** é o **NULLIF**, que age no operando e não no fluxo:

```

SELECT n.valor_icms / NULLIF(n.valor_total, 0) AS carga FROM dbo.nfe AS n;

```

7.4 COALESCE e NULLIF são CASE disfarçados

Isto explica o comportamento das duas funções e vale memorizar:

Escrita curta	Equivalente em CASE
NULLIF(a, b)	CASE WHEN a = b THEN NULL ELSE a END
COALESCE(a, b, c)	CASE WHEN a IS NOT NULL THEN a WHEN b IS NOT NULL THEN b ELSE c END
IIF(cond, x, y)	CASE WHEN cond THEN x ELSE y END

IIF (SQL Server 2012+) é apenas açúcar sintático para o **CASE** de dois ramos — útil em expressões curtas:

```

SELECT n.chave_acesso,
       IIF(n.id_destinatario IS NULL, 'CONSUMIDOR', 'IDENTIFICADO') AS destinatario,
       CHOOSE(DATEPART(QUARTER, n.data_emissao), '1o TRI', '2o TRI', '3o TRI', '4o TRI') AS
trimestre
    FROM dbo.nfe AS n
   WHERE n.id_nfe <= 10;

```

CHOOSE(n, a, b, c, ...) devolve o n-ésimo item da lista — atalho elegante para traduzir códigos sequenciais.

7.5 CASE fora da lista de seleção

No ORDER BY — para ordenação por prioridade que não é alfabética nem numérica:

```

SELECT a.numero_auto,
       a.situacao,
       FORMAT(a.valor_principal + a.valor_multa, 'N2', 'pt-BR') AS valor
    FROM dbo.auto_infracao AS a

```

```
ORDER BY CASE a.situacao
            WHEN 'INSCRITO DIVIDA ATIVA' THEN 1
            WHEN 'IMPUGNADO'           THEN 2
            WHEN 'JULGADO PROCEDENTE'  THEN 3
            WHEN 'LAVRADO'              THEN 4
            WHEN 'PAGO'                 THEN 5
            ELSE 6
END,
a.valor_principal DESC;
```

Sem o **CASE**, a ordenação alfabética colocaria **IMPUGNADO** antes de **INSCRITO**, o que não corresponde à ordem processual.

No **ORDER BY**, para controlar a posição dos nulos (o T-SQL não tem **NULLS LAST**):

```
SELECT razao_social, faturamento_declarado
FROM   dbo.contribuinte
ORDER BY CASE WHEN faturamento_declarado IS NULL THEN 1 ELSE 0 END,
          faturamento_declarado DESC;
```

No **WHERE** — filtro que muda conforme uma condição (use com parcimônia; costuma ser mais claro com **OR**):

```
DECLARE @somente_grandes BIT = 1;

SELECT c.razao_social, c.faturamento_declarado
FROM   dbo.contribuinte AS c
WHERE  1 = CASE WHEN @somente_grandes = 1 AND c.faturamento_declarado > 4800000 THEN 1
               WHEN @somente_grandes = 0 THEN 1
               ELSE 0
END;
```

No **GROUP BY** — agrupar por faixa em vez de por valor. Note que a expressão precisa ser **repetida** no **GROUP BY**, porque o alias da lista de seleção ainda não existe naquele passo (ordem lógica de processamento, Módulo 9, seção 7):

```
SELECT CASE WHEN i.valor_total_item < 1000 THEN 'ATE 1 MIL'
            WHEN i.valor_total_item < 10000 THEN '1 A 10 MIL'
            WHEN i.valor_total_item < 100000 THEN '10 A 100 MIL'
            ELSE 'ACIMA DE 100 MIL'
END AS faixa_de_valor,
COUNT(*) AS qtd_itens,
FORMAT(SUM(i.valor_total_item), 'N2', 'pt-BR') AS valor_total
FROM   dbo.nfe_item AS i
GROUP BY CASE WHEN i.valor_total_item < 1000 THEN 'ATE 1 MIL'
              WHEN i.valor_total_item < 10000 THEN '1 A 10 MIL'
              WHEN i.valor_total_item < 100000 THEN '10 A 100 MIL'
              ELSE 'ACIMA DE 100 MIL'
END
ORDER BY MIN(i.valor_total_item);
```

Alternativa mais legível: calcular a faixa em uma CTE e agrupar pelo alias na consulta externa. Prefira essa forma quando a expressão for longa.

7.6 Tradução de códigos fiscais — o uso mais frequente

Códigos são compactos para o sistema e ilegíveis para o leitor do relatório. **CASE** é o tradutor:

```

SELECT TOP (20)
    i.cst_icms,
    CASE i.cst_icms
        WHEN '000' THEN 'Tributada integralmente'
        WHEN '020' THEN 'Com reducao de base de calculo'
        WHEN '040' THEN 'Isenta'
        WHEN '041' THEN 'Nao tributada'
        WHEN '060' THEN 'ICMS cobrado anteriormente por substituicao'
        WHEN '101' THEN 'Simples Nacional com permissao de credito'
        WHEN '102' THEN 'Simples Nacional sem permissao de credito'
        ELSE 'CST nao mapeado: ' + i.cst_icms
    END
    AS situacao_tributaria,
    LEFT(i.cfop, 1) AS grupo_cfop,
    CASE LEFT(i.cfop, 1)
        WHEN '1' THEN 'Entrada estadual'
        WHEN '2' THEN 'Entrada interestadual'
        WHEN '3' THEN 'Entrada do exterior'
        WHEN '5' THEN 'Saida estadual'
        WHEN '6' THEN 'Saida interestadual'
        WHEN '7' THEN 'Saida para o exterior'
        ELSE 'CFOP invalido'
    END
    AS natureza_da_operacao,
    COUNT(*) OVER (PARTITION BY i.cst_icms) AS itens_no_mesmo_cst
FROM    dbo.nfe_item AS i
ORDER BY i.id_item;

```

O que observar no ELSE: em vez de um genérico 'OUTROS', o ramo devolve 'CST nao mapeado: ' + i.cst_icms. Assim, um código novo ou inesperado **aparece no relatório** em vez de se esconder atrás de um rótulo genérico. Em trabalho fiscal, o desconhecido precisa ser visível.

Etapa 8 — Saída pronta para relatório

8.1 Agregação condicional: o pivô manual

CASE dentro de função agregada transforma linhas em colunas — a técnica mais útil de todo este roteiro:

```

SELECT c.cnpj,
    UPPER(TRIM(c.razao_social)) AS razao_social,
    COUNT(*) AS total_notas,
    SUM(CASE WHEN n.tipo_operacao = '1' THEN 1 ELSE 0 END) AS saidas,
    SUM(CASE WHEN n.tipo_operacao = '0' THEN 1 ELSE 0 END) AS entradas,
    COUNT(CASE WHEN n.situacao = 'CANCELADA' THEN 1 END) AS canceladas,
    COUNT(CASE WHEN n.situacao = 'DENEGADA' THEN 1 END) AS denegadas,
    FORMAT(SUM(CASE WHEN n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
        THEN n.valor_total ELSE 0 END), 'N2', 'pt-BR') AS valor_saidas,
    FORMAT(CAST(COUNT(CASE WHEN n.situacao = 'CANCELADA' THEN 1 END) AS DECIMAL(12,4))
        / NULLIF(COUNT(*), 0), 'P2', 'pt-BR') AS pct_cancelamento
FROM    dbo.nfe AS n
    INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
GROUP BY c.cnpj, c.razao_social
HAVING COUNT(*) >= 100
ORDER BY COUNT(CASE WHEN n.situacao = 'CANCELADA' THEN 1 END) * 1.0 / COUNT(*) DESC;

```

O que observar — duas formas equivalentes com uma diferença sutil:

Escrita	Comportamento
<code>SUM(CASE WHEN cond THEN 1 ELSE 0 END)</code>	soma 1 quando a condição é verdadeira e 0 quando não é
<code>COUNT(CASE WHEN cond THEN 1 END)</code>	conta apenas quando verdadeira; sem ELSE , o restante vira <code>NULL</code> , e <code>COUNT</code> ignora nulos

Ambas produzem o mesmo número. **O erro fatal** é escrever `COUNT(CASE WHEN cond THEN 1 ELSE 0 END)`: com o `ELSE 0`, todas as linhas passam a ter valor não nulo e `COUNT` devolve o total de linhas, não o total de ocorrências.

8.2 STRING_AGG: transformar linhas em uma lista

```
SELECT c.cnpj,
       UPPER(TRIM(c.razao_social))           AS razao_social,
       COUNT(DISTINCT i.cfop)                 AS qtd_cfop_distintos,
       STRING_AGG(i.cfop, ', ') WITHIN GROUP (ORDER BY i.cfop) AS cfop_utilizados
FROM   dbo.contribuinte AS c
       INNER JOIN dbo.nfe      AS n ON n.id_emitente = c.id_contribuinte
       INNER JOIN dbo.nfe_item AS i ON i.id_nfe      = n.id_nfe
WHERE  n.situacao = 'AUTORIZADA'
GROUP BY c.cnpj, c.razao_social
HAVING COUNT(DISTINCT i.cfop) >= 4
ORDER BY qtd_cfop_distintos DESC;
```

O que observar: `STRING_AGG` (SQL Server 2017+) concatena os valores de um grupo com um separador; `WITHIN GROUP (ORDER BY ...)` define a ordem dentro da lista. Sem ele, a ordem é indefinida — e um relatório em que a mesma consulta produz listas em ordens diferentes a cada execução não é reproduzível.

Para eliminar repetições dentro da lista, agregue o distinto antes:

```
WITH cfop_por_contribuinte AS (
    SELECT DISTINCT n.id_emitente, i.cfop
    FROM   dbo.nfe AS n INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe
    WHERE  n.situacao = 'AUTORIZADA'
)
SELECT c.cnpj,
       STRING_AGG(cp.cfop, ', ') WITHIN GROUP (ORDER BY cp.cfop) AS cfop_distintos
FROM   cfop_por_contribuinte AS cp
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = cp.id_emitente
GROUP BY c.cnpj;
```

8.3 Indicadores com semáforo

```
WITH base AS (
    SELECT n.id_emitente,
           COUNT(*)           AS qtd,
           SUM(n.valor_total) AS total,
           SUM(n.valor_icms)  AS icms,
           SUM(n.valor_icms) / NULLIF(SUM(n.valor_total), 0) AS carga
    FROM   dbo.nfe AS n
    WHERE  n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
    GROUP BY n.id_emitente
)
SELECT SUBSTRING(c.cnpj,1,2) + '.' + SUBSTRING(c.cnpj,3,3) + '.' +
       SUBSTRING(c.cnpj,6,3) + '/' + SUBSTRING(c.cnpj,9,4) + '-' +
       SUBSTRING(c.cnpj,13,2)           AS [CNPJ],
       UPPER(TRIM(c.razao_social))      AS [Contribuinte],
```

```

        c.regime_tributario                AS [Regime],
        FORMAT(b.qtd, 'N0', 'pt-BR')       AS [Notas],
        FORMAT(b.total, 'N2', 'pt-BR')     AS [Saidas],
        FORMAT(b.carga, 'P2', 'pt-BR')     AS [Carga],
        CASE WHEN c.regime_tributario <> 'NORMAL' THEN 'NAO SE APLICA'
            WHEN b.carga < 0.04 THEN 'CRITICO'
            WHEN b.carga < 0.10 THEN 'ATENCAO'
            ELSE 'REGULAR'
        END                                AS [Situacao],
        REPLICATE('#', CASE WHEN c.regime_tributario <> 'NORMAL' THEN 0
            WHEN b.carga < 0.04 THEN 5
            WHEN b.carga < 0.10 THEN 3
            ELSE 1 END)                    AS [Risco]
FROM    base AS b
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = b.id_emitente
WHERE   b.total > 100000
ORDER BY b.carga;

```

O que observar: `REPLICATE` desenha uma barra de risco em texto puro, legível em qualquer meio — inclusive em um `.txt` anexado ao processo. O ramo `'NAO SE APLICA'` para o Simples Nacional evita o falso positivo mais comum desse indicador: empresas do Simples destacam ICMS reduzido por natureza, e apareceriam todas como "críticas".

8.4 O carimbo de extração

Toda listagem que instrui procedimento fiscal deve dizer **quem, quando e de onde** foi extraída:

```

SELECT FORMAT(GETDATE(), 'dd/MM/yyyy HH:mm:ss') AS extraido_em,
        SYSTEM_USER                             AS usuario,
        @@SERVERNAME                             AS servidor,
        DB_NAME()                                AS banco,
        @@VERSION                                 AS versao_do_sgbd;

```

Cole esse cabeçalho no topo do relatório. É o que permite, meses depois, reproduzir a apuração exatamente como ela foi feita — a exigência de reprodutibilidade discutida no Módulo 10, seção 15.

Etapa 9 — Consulta integradora

O produto final do laboratório: uma listagem de malha fiscal completa, formatada e pronta para anexar à ordem de serviço.

```

DECLARE @periodo CHAR(6) = '202501';
DECLARE @ini      DATE   = '2025-01-01';
DECLARE @fim      DATE   = '2025-02-01';

WITH omissoes AS (
    SELECT n.id_emitente,
           COUNT(*)      AS qtd_documentos,
           SUM(n.valor_total) AS valor_omitido,
           SUM(n.valor_icms) AS icms_potencial,
           MIN(n.data_emissao) AS primeira,
           MAX(n.data_emissao) AS ultima
    FROM    dbo.nfe AS n
           INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
    WHERE   n.situacao = 'AUTORIZADA'
           AND n.tipo_operacao = '1'
           AND n.data_emissao >= @ini AND n.data_emissao < @fim
           AND c.regime_tributario = 'NORMAL'
           AND NOT EXISTS (SELECT 1 FROM dbo.efdc100 AS e
                           WHERE e.chave_acesso = n.chave_acesso
                           AND e.id_contribuinte = n.id_emitente)
)

```



```

        AND e.ind_oper      = '1'
        AND e.cod_situacao  = '00')
    GROUP BY n.id_emitente
),
socios AS (
    SELECT s.id_contribuinte,
           STRING_AGG(UPPER(TRIM(s.nome)), ' | ')
           WITHIN GROUP (ORDER BY UPPER(TRIM(s.nome))) AS quadro_societario
    FROM   dbo.socio_administrador AS s
    GROUP BY s.id_contribuinte
)
SELECT ROW_NUMBER() OVER (ORDER BY o.icms_potencial DESC) AS [#],
       SUBSTRING(c.cnpj,1,2) + '.' + SUBSTRING(c.cnpj,3,3) + '.' +
       SUBSTRING(c.cnpj,6,3) + '/' + SUBSTRING(c.cnpj,9,4) + '-' +
       SUBSTRING(c.cnpj,13,2) AS [CNPJ],
       UPPER(TRIM(c.razao_social)) AS [Contribuinte],
       m.nome AS [Municipio],
       COALESCE(m.regiao_fiscal, 'FORA DA PB') AS [Regiao],
       FORMAT(o.qtd_documentos, 'N0', 'pt-BR') AS [Docs],
       FORMAT(o.valor_omitido, 'N2', 'pt-BR') AS [Valor Omitido],
       FORMAT(o.icms_potencial, 'N2', 'pt-BR') AS [ICMS Potencial],
       CONCAT(FORMAT(o.primeira, 'dd/MM'), ' a ', FORMAT(o.ultima, 'dd/MM/yyyy')) AS [Periodo],
       CASE WHEN o.icms_potencial >= 100000 THEN 'ALTA'
            WHEN o.icms_potencial >= 20000 THEN 'MEDIA'
            ELSE 'BAIXA'
       END AS [Prioridade],
       COALESCE(s.quadro_societario, '(sem socio cadastrado)') AS [Quadro Societario],
       CONCAT('Extraido em ', FORMAT(GETDATE(), 'dd/MM/yyyy'),
              ' por ', SYSTEM_USER, ' - competencia ', @periodo) AS [Origem]
FROM   omissoes AS o
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = o.id_emitente
       INNER JOIN dbo.municipio   AS m ON m.id_municipio   = c.id_municipio
       LEFT JOIN  socios           AS s ON s.id_contribuinte = c.id_contribuinte
WHERE  o.icms_potencial > 1000
ORDER BY o.icms_potencial DESC;

```

O que observar:

1. Toda a **seleção** (`WHERE`, `NOT EXISTS`, `GROUP BY`) opera sobre o dado **bruto**; a formatação só aparece na lista de seleção final.
2. `ORDER BY o.icms_potencial DESC` usa a coluna numérica, não o texto formatado.
3. `ROW_NUMBER()` numera as linhas do relatório — e é calculado depois da ordenação declarada no `OVER`.
4. O `COALESCE` no quadro societário evita a linha vazia que denunciaria falta de cadastro como se fosse falha do relatório.

Etapa 10 — Desempenho e boas práticas

10.1 Meça o custo da formatação

```

SET STATISTICS TIME ON;

-- (a) sem formatacao
SELECT COUNT(*) FROM (SELECT valor_total FROM dbo.nfe_item x
                        INNER JOIN dbo.nfe n ON n.id_nfe = x.id_nfe) AS t;

-- (b) com CONVERT
SELECT COUNT(*) FROM (SELECT CONVERT(VARCHAR(30), CAST(valor_total_item AS MONEY), 1) AS v
                        FROM dbo.nfe_item) AS t;

-- (c) com FORMAT
SELECT COUNT(*) FROM (SELECT FORMAT(valor_total_item, 'N2', 'pt-BR') AS v
                        FROM dbo.nfe_item) AS t;

SET STATISTICS TIME OFF;

```

Sobre as ~31 mil linhas de `nfe_item`, a diferença já é perceptível; sobre milhões, é a diferença entre segundos e minutos.

10.2 As sete regras do laboratório

#	Regra	Motivo
1	Formate na última consulta, nunca nas intermediárias	formatação quebra junção, ordenação e agregação posteriores
2	Ordene pela coluna bruta , exiba a formatada	texto ordena como texto: <code>'9.000,00' > '10.000,00'</code>
3	Nunca aplique função à coluna do <code>WHERE</code>	destrói o uso de índice (predicado não <i>sargable</i>)
4	<code>CAST</code> para <code>DECIMAL</code> antes de dividir contagens	divisão inteira trunca e o percentual sai zero
5	Use <code>DECIMAL</code> , jamais <code>FLOAT</code> , para dinheiro	<code>FLOAT</code> acumula erro binário indefensável em auto de infração
6	Prefira <code>CONVERT</code> + <code>REPLACE</code> a <code>FORMAT</code> em grande volume	<code>FORMAT</code> é CLR e custa uma ordem de grandeza a mais
7	Normalize CPF/CNPJ antes de cruzar, formate depois	máscara heterogênea faz o cruzamento não encontrar nada

10.3 Uma última advertência sobre exportação

Ao exportar para CSV que será reimportado por outro sistema, exporte o dado **limpo** (CNPJ com 14 dígitos, data em `AAAAMMDD`, valor com ponto decimal). A máscara é para o leitor humano; o sistema de destino quase sempre a rejeita — e o Excel, como visto no Módulo 10, transforma `27712979000187` em notação científica e `06.123.456/0001-99` em texto truncado. Quando o destino é o Excel, importe pelo Power Query definindo as colunas como texto.

Etapa 11 — Tabelas-resumo

11.1 Numéricas e de arredondamento

Função	Faz	Exemplo
<code>ROUND(x, n)</code>	arredonda para n casas	<code>ROUND(13640.16, 1) → 13640.20</code>
<code>ROUND(x, n, 1)</code>	trunca em n casas	<code>ROUND(13640.16, 1, 1) → 13640.10</code>
<code>CEILING(x)</code> / <code>FLOOR(x)</code>	teto / piso	<code>CEILING(13640.16) → 13641</code>
<code>ABS(x)</code>	valor absoluto	<code>ABS(-25.50) → 25.50</code>
<code>NULLIF(a, b)</code>	<code>NULL</code> se <code>a = b</code>	evita divisão por zero
<code>CAST(x AS DECIMAL(15,2))</code>	converte com precisão	obrigatório antes de dividir contagens

11.2 Datas

Função	Faz
CONVERT(VARCHAR, data, 103 112 23)	formata com estilo (dd/mm/aaaa, aaaammdd, ISO)
FORMAT(data, 'dd/MM/yyyy', 'pt-BR')	formata com máscara e cultura
DATEDIFF(unidade, ini, fim)	diferença entre datas
DATEADD(unidade, n, data)	soma período
EOMONTH(data [, n])	último dia do mês (n meses à frente/atrás)
DATEPART / DATENAME	parte numérica / nome (depende do idioma)
GETDATE() / SYSDATETIME()	data e hora do servidor

11.3 Strings e máscaras

Função	Faz
LEN / DATALENGTH	comprimento (ignora espaço à direita) / bytes
TRIM / LTRIM / RTRIM	remove espaços
UPPER / LOWER	caixa
LEFT / RIGHT / SUBSTRING	recortes
REPLACE(t, de, para)	substitui todas as ocorrências
STUFF(t, pos, qtd, novo)	insere/substitui em posição (com qtd = 0, só insere)
REPLICATE(t, n)	repete texto — preenchimento e barras
CONCAT / CONCAT_WS	concatena ignorando nulos
CHARINDEX(busca, t)	posição da primeira ocorrência
STRING_AGG(col, sep) WITHIN GROUP (ORDER BY ...)	agrega linhas em uma lista

11.4 Máscaras do domínio fiscal

Campo	Expressão
CNPJ	SUBSTRING(cnpj,1,2)+'.'+SUBSTRING(cnpj,3,3)+'.'+SUBSTRING(cnpj,6,3)+'/'+SUBSTRING(cnpj,9,4)+'-' +SUBSTRING(cnpj,13,2)
CPF	SUBSTRING(cpf,1,3)+'.'+SUBSTRING(cpf,4,3)+'.'+SUBSTRING(cpf,7,3)+'-' +SUBSTRING(cpf,10,2)
Limpar máscara	REPLACE(REPLACE(REPLACE(TRIM(x),'.',''),'-',''),' ','')
Recompor zeros	RIGHT('0000000000' + digitos, 11) (CPF) / RIGHT(REPLICATE('0',14) + digitos, 14) (CNPJ)
NCM	STUFF(STUFF(ncm, 7, 0, '.'), 5, 0, '.') → 2204.21.00
CFOP	STUFF(cfop, 2, 0, '.') → 5.405
Período AAAAMM	CONVERT(VARCHAR(6), data, 112)
Competência por extenso	FORMAT(CONVERT(DATE, periodo + '01'), 'MMM/yyyy', 'pt-BR')

11.5 CASE em uma página

```
-- Forma simples: compara por igualdade
CASE expressao WHEN v1 THEN r1 WHEN v2 THEN r2 ELSE rn END

-- Forma pesquisada: condições booleanas quaisquer
CASE WHEN cond1 THEN r1 WHEN cond2 THEN r2 ELSE rn END
```

Regra	Consequência
Avaliação sequencial	ordene da condição mais restritiva para a mais ampla
Sem ELSE → NULL	proposital em COUNT(CASE ...), esquecimento em relatório
Tipo de retorno único	converta explicitamente os ramos (FORMAT, CAST)
É expressão, não comando	cabe em SELECT, WHERE, GROUP BY, HAVING, ORDER BY, agregações
IIF, COALESCE, NULLIF	são CASE abreviados

Etapa 12 — Exercícios

As respostas comentadas estão em [gabarito_roteiro_formatacao_tsq1.md](#). Resolva antes de consultar. Todos os exercícios usam o banco [curso_integridade_fiscal](#) com os scripts 01 a 09 executados.

Bloco A — Números e arredondamento

- A1.** Para as dez NF-e autorizadas de maior valor, exiba a chave de acesso, o valor total arredondado ao real (sem casas decimais e como número inteiro), o valor truncado ao real e a diferença entre os dois.
- A2.** Calcule, para a tabela `nfe_item` inteira, três somatórios do ICMS a 18%: (a) somando antes de arredondar; (b) arredondando cada item antes de somar; (c) truncando cada item antes de somar. Apresente as três colunas e as duas diferenças. Em duas linhas de comentário, indique qual das três é a base adequada para conferir escrituração e por quê.
- A3.** Para cada regime tributário, calcule o percentual de notas canceladas sobre o total, com duas casas decimais. Garanta que o resultado não seja zero por divisão inteira.
- A4.** Exiba o valor total das dez maiores notas em três formatos na mesma linha: número simples com duas casas, número no padrão brasileiro (13.640,16) e moeda (R\$ 13.640,16).
- A5.** (Desafio) Monte uma coluna de texto com 18 posições, alinhada à direita, contendo o valor total formatado no padrão brasileiro, e outra com a razão social ocupando exatamente 45 posições à esquerda. Verifique o alinhamento com a saída em texto (Ctrl+T).

Bloco B — Datas

- B1.** Para as vinte primeiras ordens de serviço, exiba: número da OS, data de abertura em dd/mm/aaaa, competência de abertura em AAAAMM, mês por extenso em português e dia da semana.
- B2.** Para cada ordem de serviço ainda sem conclusão, calcule os dias corridos desde a abertura e classifique em NO PRAZO (até 90 dias), ATENCAO (91 a 180) e CRITICA (acima de 180).
- B3.** Liste os períodos de apuração distintos existentes em `efd_c100` e, ao lado de cada um, o primeiro dia do período, o último dia e a competência por extenso (jan/2025).
- B4.** Para cada auto de infração, exiba a data de lavratura por extenso no formato 07 de abril de 2025, e o prazo final de impugnação (30 dias corridos após a lavratura) no formato dd/mm/aaaa.
- B5.** Escreva duas versões da consulta que soma o valor das NF-e autorizadas emitidas em março de 2025: uma usando YEAR() e MONTH(), outra usando intervalo de datas. Compare os planos de execução (Ctrl+M) e explique a diferença.

Bloco C — Conversões e qualidade do dado

- C1.** Conte quantos registros de `socio_administrador` têm CPF diretamente conversível para número e quantos não têm. Liste cinco exemplos de cada grupo.
- C2.** Liste todos os sócios cujo CPF, depois de removida a máscara, tenha menos de 11 dígitos. Explique, em uma linha, a causa provável desse defeito.
- C3.** Converta o `periodo_apuracao` de `efd_c100` em data e apure o total escriturado por trimestre, exibindo o trimestre como `1o TRI/2025`.
- C4.** Demonstre a diferença entre `CAST(valor_total AS INT)` e `ROUND(valor_total, 0)` em cinco notas cujo valor tenha centavos acima de cinquenta.
- C5.** (Desafio) Escreva uma consulta que identifique registros de `socio_administrador` cujo CPF, após a limpeza, **não** tenha dígito verificador válido. Implemente o cálculo do DV com aritmética T-SQL sobre os dígitos.

Bloco D — CASE

- D1.** Traduza o campo `cod_situacao` de `efd_c100` para texto legível (`00` = documento regular, `02` = cancelado, demais = '`Codigo nao mapeado: XX`') e conte os registros de cada tipo.
- D2.** Classifique todos os itens de `nfe_item` por faixa de alíquota: `ISENTO/NAO TRIBUTADO` (alíquota nula ou zero), `REDUZIDA` (até 7%), `INTERESTADUAL` (acima de 7% até 12%), `INTERNA` (acima de 12%). Apresente a quantidade e o valor total por faixa.
- D3.** Liste os autos de infração ordenados pela fase processual — `INSCRITO DIVIDA ATIVA`, `IMPUGNADO`, `JULGADO PROCEDENTE`, `LAVRADO`, `PAGO`, `JULGADO IMPROCEDENTE` — e, dentro de cada fase, pelo valor decrescente.
- D4.** Reescreva usando `IIF` e depois usando `CASE` a coluna que indica se a NF-e foi emitida a consumidor não identificado. Diga qual das duas você usaria em material do curso e por quê.
- D5.** Explique, em suas palavras e com um exemplo próprio sobre a base, por que a consulta abaixo falha e como corrigi-la:

```
SELECT CASE WHEN c.faturamento_declarado IS NULL THEN 'NAO INFORMADO'
          ELSE c.faturamento_declarado END AS faturamento
FROM   dbo.contribuinte AS c;
```

- D6.** Produza uma tabela cruzada por município com uma coluna para cada situação de NF-e (autorizada, cancelada, denegada, inutilizada), usando agregação condicional. Acrescente o percentual de cancelamento com duas casas.

Bloco E — Máscaras de CPF e CNPJ

- E1.** Exiba os vinte primeiros contribuintes com o CNPJ formatado, obtendo a máscara por dois caminhos diferentes (`SUBSTRING` e `STUFF`) e confirme que os resultados coincidem.
- E2.** Normalize o CPF de todos os sócios (remover máscara, remover espaços e recompor zeros à esquerda) e exiba, lado a lado, o valor original e o normalizado, apenas para os registros em que os dois diferem.
- E3.** Identifique as pessoas físicas que constam como sócias de três ou mais contribuintes. Exiba o CPF formatado, o nome normalizado, a quantidade de empresas e a lista das razões sociais separadas por `|`.
- E4.** Execute a consulta do E3 **sem** normalizar o CPF e compare os resultados. Escreva um parágrafo explicando o que a diferença significa para um trabalho de identificação de grupo econômico.
- E5.** Formate a chave de acesso em grupos de quatro dígitos e, na mesma linha, extraia dela a UF, a competência (`AAMM`), o CNPJ do emitente **já mascarado** e o número da NF-e sem zeros à esquerda.

Bloco F — Relatório integrador

- F1.** Produza o relatório final de malha fiscal de janeiro de 2025 contendo: número sequencial, CNPJ mascarado, razão social normalizada, município, região fiscal (com `FORA DA PB` quando nula), quantidade de documentos omitidos, valor e ICMS formatados

no padrão brasileiro, período coberto (dd/mm a dd/mm/aaaa), prioridade por faixa de ICMS e o carimbo de extração. Ordene pela prioridade e pelo ICMS.

F2. Acrescente ao relatório do F1 uma coluna com o quadro societário (nomes normalizados, separados por |) e outra que sinalize **SOCIO EM COMUM COM OUTRO CONTRIBUINTE** quando algum sócio do contribuinte também for sócio de outra empresa da base.

F3. Elabore uma versão do mesmo relatório destinada à **exportação para outro sistema**: sem máscaras, datas em AAAAMDD, valores com ponto decimal e sem separador de milhar. Justifique cada escolha em comentários no próprio script.

F4. (Desafio) Escreva uma consulta que produza, em uma única coluna de texto, o parágrafo de um ofício de intimação, no formato:

"Fica o contribuinte ALFA DISTRIBUIDORA DE BEBIDAS EIRELI, inscrito no CNPJ sob o nº 27.712.979/0001-87, estabelecido no município de ALAGOA GRANDE, intimado a apresentar, no prazo de 30 (trinta) dias contados de 29 de julho de 2026, os arquivos da EFD ICMS/IPI referentes à competência janeiro/2025, tendo em vista a constatação de 12 (doze) documentos fiscais emitidos e não escriturados, no valor total de R\$ 145.320,55."

Todos os dados variáveis devem vir da base, com a formatação adequada.

Etapa 13 — Referências

MICROSOFT. **FORMAT (Transact-SQL), CAST and CONVERT (Transact-SQL), STUFF, STRING_AGG, CASE (Transact-SQL), TRY_CAST, TRY_CONVERT**. Microsoft Learn — Transact-SQL Reference.

PETKOVIC, Dusan. **Microsoft SQL Server 2019: A Beginner's Guide**. 7. ed. McGraw-Hill, 2020. Capítulo 5 (funções de sistema) e Capítulo 6 (a expressão CASE).

ELMASRI, Ramez; NAVATHE, Shamkant B. **Fundamentals of Database Systems**. 7. ed. Pearson, 2015. Seção 6.3 — expressões e renomeação na lista de seleção.

BRASIL. **Guia Prático da EFD ICMS/IPI** — leiaute dos registros C100 e C170 e formato dos campos numéricos e de data.

BRASIL. **Manual de Orientação do Contribuinte — NF-e**, Anexo I: composição da chave de acesso.

Material didático produzido para o Curso de Banco de Dados Relacional aplicado à Fiscalização Tributária — Secretaria de Estado da Fazenda da Paraíba.